Introduccion a la Ingenieria de Software

Software

Es un set de programas y la documentación que acompaña, no es únicamente código, por lo que saber programar no alcanza para hacer software.

1 Apple (881.980 millones de dólares)

2 Microsoft (803.090 millones de dólares)

3 Amazon (739.460 millones de dólares)

4 Alphabet (711.940 millones de dólares)

5 Berkshire Hathaway (536.530 millones de dólares)

6 Johnson & Johnson (396.210 millones de dólares)

7 Alibaba (387.610 millones de dólares)

8 Facebook (378.050 millones de dólares)

9 JPMorgan Chase & Co (368.560 millones de dólares)

10 Tencent Holdings (353.670 millones de dólares)



1 – La destrucción del Mariner I (1962). 18,5 millones de dólares.

El del Mariner I, una sonda espacial que se dirigía a Venus, se desvió de la trayectoria de vuelo prevista poco después del lanzamiento. Desde control se destruyó la sonda a los 293 segundos del despegue. La causa fue una fórmula manuscrita que se programó incorrectamente.

2 – La catástrofe del Hartford Coliseum (1978). 70 millones de dólares.

Apenas unas horas después de que miles de aficionados abandonaron el Hartford Coliseum, el techo se derrumbó por el peso de la nieve. La causa: cálculo incorrecto introducido en el software CAD utilizado para diseñar el coliseo.

3 – El gusano de Morris (1988). 100 millones de dólares.

El estudiante de posgrado Robert Tappan Morris fue condenado por el primer ataque con "gusanos" a gran escala en Internet. Los costos de limpiar el desastre se cifran en 100 millones de dólares. Morris, es hoy profesor en MIT.

4 – Error de cálculo de Intel (1994). 475 millones de dólares.

Un profesor de matemáticas descubrió y difundió que había un fallo en el procesador Pentium de Intel. La sustitución de chips costó a Intel 475 millones.

5 – Explosión del cohete Arian (1996). 500 millones de dólares.

En el 1996, el cohete Ariane 5 de la Agencia Espacial Europea estalló. El Ariane explotó porque un número real de 64 bits (coma flotante) relacionado con la velocidad se convirtió en un entero de 16 bits. Te dejo un post que explica lo sucedido.

6 – Mars Climate Orbiterm (1999). 655 millones de dólares.

En 1999 los ingenieros de la NASA perdieron el contacto con la Mars Climate Orbiter en su intento que orbitase en Marte. La causa, un programa calculaba la distancia en unidades inglesas (pulgadas, pies y libras), mientras que otro utilizó unidades métricas.

7 – El error en los frenos de los Toyota (2010). 3 billones de dólares.

Toyota retiró más de 400.000 de sus vehículos híbridos en 2010, por un problema software, que provocaba un retraso en el sistema anti-bloqueo de frenos. Se estima entre sustituciones y demandas el error le costó a Toyota 3 billones de dólares.

8 – Las migraciones por el año 2000. 296,7 billones de dólares.

Se esperaba que el bug Y2K paralizase al mundo a la medianoche del 1 de enero 2000, ya que mucho software no había sido previsto para trabajar con el año 2000. El mundo no se acabó, pero se estima que se gastaron 296,7 billones de dólares para mitigar los daños.

Tipos de software

- **System Software** → se encarga de controlar y gestionar los recursos del hardware de una computadora. Ej: Sistemas operativos: Linux, Windows, etc

- **Utilitarios** → subconjunto de system software que proporciona utilidades o herramientas adicionales para mejorar la función del SO. Ej: Antivirus, Compresores
    - *Objetivo:* Mejorar y optimizar el rendimiento del SO y de hardware de la pc.

- **Software de aplicación** → usado para realizar tareas específicas en una computadora, más enfocada en satisfacer las necesidades de los usuarios.

Razones para no comparar software y manufactura

- Es menos predecible
- No hay producción en masa, casi ninguno es igual a otro
- No todas las fallas son errores
- No se gasta
- No va de la mano con las leyes de la física

Problemas al desarrollar software

- La versión final no satisface las necesidades del cliente
- Difícil de extender/adaptar
- Mala documentacion
- Mala calidad
- Más tiempo y costo del presupuestado → más común

Proceso de software

Conjunto estructurado de actividades que, a raíz de un conjunto de entradas producen una salida.

Tiene como objetivo obtener un producto o servicio de software a partir de distintas entradas.
Definiciones:

- **IEEE:** Es una secuencia de pasos para un propósito dado.
- **CMM:** Conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados

Proceso Definido

Basado en modelo industrial, un conjunto de entradas usadas en un conjunto de actividades van a producir el mismo resultado o salida siempre que las entradas sean las mismas.
Caracteristicas:

- Difícil de aplicar al software, ya que no es predecible.
- No podemos decir que el software, con ciertas entradas usadas en ciertas aplicaciones van a producir las mismas salidas.
- Repetir el proceso indefinidamente y obtener los mismos resultados

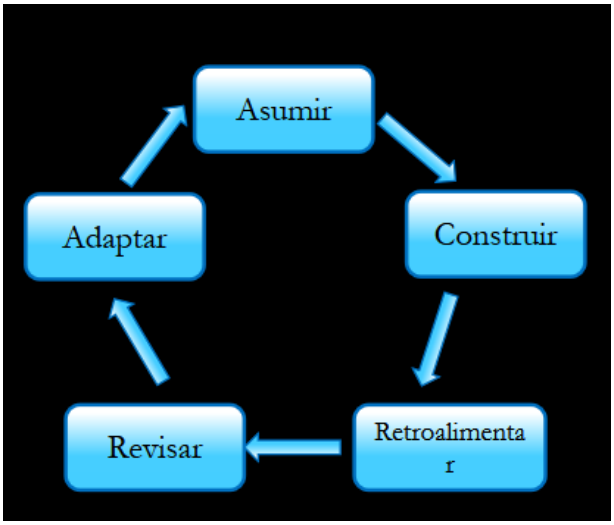
Proceso Empirico

Viene del empirismo y nos centramos en la experiencia (del usuario, del negocio, de la técnica) y en el aprendizaje.
Caracteristicas:

- Las variables no pueden ser controladas en su totalidad, hay que adaptarse a ellas para garantizar la calidad del software
- Busco generar cambios rápidos, e implementarlos sobre periodos, para que si no sirve no duela tirarlo.
- La experiencia solo cuenta en ese momento, en ese proyecto, para ese cliente y para el mismo equipo.

Patrón de conocimiento

- **Hipotesis (ASUMIR)**
- **Construcción (CONSTRUIR):**
- **Retrospectiva (RETROALIMENTAR):** Inspeccionar y analizar para hacer la retrospectiva
- **Revisamos (REVISAR):** Comparar el resultado de la construcción con la hipótesis inicial.
- **ADAPTAR**



Esto es UNA repetición, o UN CICLO DE VIDA específico

Ciclo de vida

Define cuales son las etapas y el orden de cada una. Determina cuánto de cada una de esas actividades vas a realizar a la vez, nos dice cómo usar el proceso en contexto del proyecto. Son la serie de pasos a través de los cuales un proceso o producto progresa.

En un proyecto

Es una representación de un proceso. Gráfica una descripción del proceso desde cierta perspectiva.

De un producto

El ciclo de vida de un proyecto termina cuando genera un producto (el ciclo de este no termina, prospera mientras exista, ya que necesitará mantenimiento)

Tipos de ciclos de vida:

- **Secuencial:** ejecutan una etapa después de otra sin retorno.
    - Ejemplo: Primero se definen los requisitos, luego se hace el diseño, después el desarrollo, las pruebas y finalmente la entrega
- **Iterativo Incremental:** Usados por los procesos empíricos para poder adaptarse y mejorar el proceso, en cada una de las iteraciones capitalizamos lo aprendido, mejoremos la experiencia del equipo y realicemos otra iteración para incrementar el producto.
    - Hago iteraciones para ganar información y convertir esa información en un incremento que será incorporado en el sistema.
    - Ejemplo: en la primera iteración se crea el registro de gastos, en la segunda se agregan filtros y se mejora el registro, en la tercera se agregan gráficos y se mejoran los filtros.
- **Recursivo:** Usado en casos particulares, como en proyectos de alto riesgo. Toman una característica específica y en una iteración me centro en esa funcionalidad, en la otra la siguiente, etc (NO SON USADAS)
    - *En espiral*: Investigación de riesgos
    - *En B:* permite hacer una vuelta particular de cada una de las etapas e ir avanzando en los incrementos
    - Ejemplo: Se analiza un requisito, se detecta que necesita mayor detalle, se vuelve a analizar y luego se continúa.

En base a las características del proyecto se va a elegir el ciclo de vida que se va a implementar para poder llevarlo a cabo de la mejor forma posible. Siempre dependerá de qué beneficios tiene el mismo según costos y el entorno.

El **proceso** es una implementación del **ciclo de vida** que tiene un objetivo que lo guía, que es la creación de un **producto**.

Ventajas y desventajas

Secuencial

Ventajas

- Simple de organizar y entender
- Cada etapa tiene un objetivo claro
- Facilita la planificación y el seguimiento
- Permite saber en qué etapa se encuentra el proyecto

Desventajas

- Poco flexible ante cambios
- Si aparece un error en una etapa anterior, es difícil volver
- El usuario obtiene el producto recién al final
- No aprovecha el aprendizaje durante el desarrollo

Iterativo Incremental

Ventajas

- Permite adaptarse a cambios
- Permite aprender durante el desarrollo

- Cada iteración aporta información nueva
- El producto crece progresivamente mediante incrementos
- Permite incorporar el feedback y mejorar lo analizado

Desventajas

- Requiere adaptación constante
- Puede generar retrabajo
- Es más difícil saber exactamente cómo será el producto final desde el comienzo
- Requiere una buena organización entre las iteraciones

Recursivo

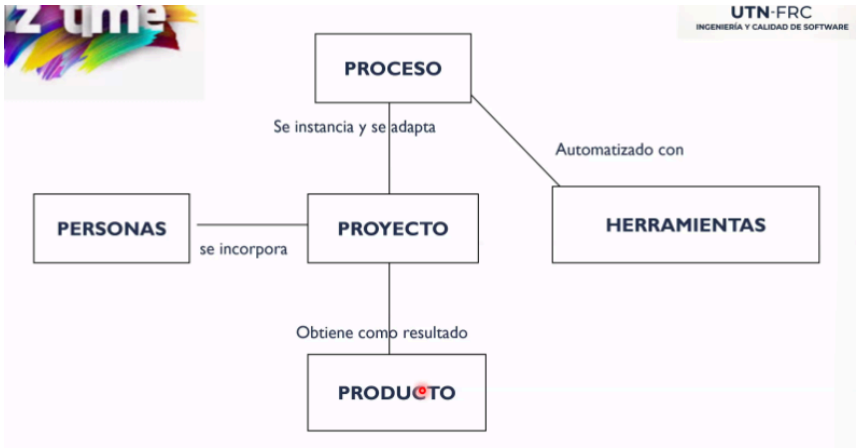
Ventajas

- Permite concentrarse profundamente en características específicas
- Permite tratar aspectos complejos o de alto riesgo
- Facilita analizar cada característica de manera detallada

Desventajas

- Puede ser más complejo de gestionar
- Puede requerir mucho tiempo
- Puede generar muchas vueltas sobre una misma característica
- No es un enfoque usado habitualmente

Proyecto de Sistemas de Información



Un **proyecto** es llevado a cabo por **personas** que implementan **herramientas** para automatizar los **procesos** y que tiene como resultado un **producto**.

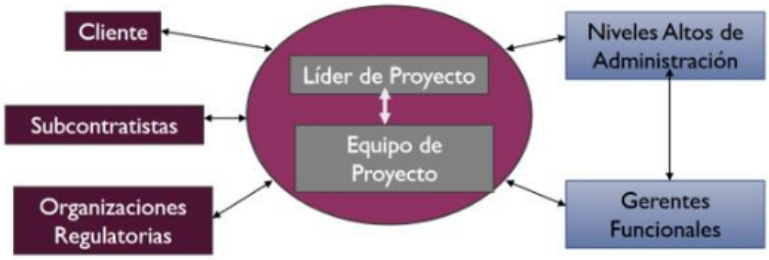
Proceso: definición abstracta que se materializa a través de los **proyectos**, se adapta a las necesidades concretas del mismo. En él se expresa en forma teórica lo que se debe hacer para hacer software y debo adaptarlo al proyecto (si es un proceso definido) o terminar de armarlo (si es un proceso empírico).
Los procesos toman como entrada requerimientos, y a través de un conjunto de actividades (guiadas y estructuradas por un objetivo) obtienen como salida un producto.

Proyecto: unidad organizativa de gestión, necesita de personas, recursos y procesos para poder existir.

Caracteristicas

- **Duración limitada:** son temporales, cuando se alcanza el objetivo, el proyecto termina. (Tiene un inicio y un fin)
- **Orientado a un objetivo:** tengo que poder definir hacia dónde voy y poder medir cuando se alcance el objetivo para poder finalizarlo. Tienen un único objetivo que tiene que ser claro, no ambiguo y alcanzable. (Esto me permite que los proyectos sean únicos)
- **Son únicos:** todos los proyectos similares que tienen características que los hacen únicos.
- **Tiene tareas relacionadas entre sí basadas en esfuerzos y recursos para alcanzar el objetivo:** las tareas dependen una de otra, tienen precedencia y asociados recursos y esfuerzos

Roles de un proyecto



- **Líder o Project Manager (PM):** se ocupa de todas las tareas de gestión que hacen que el equipo del proyecto sea guiado a lograr el objetivo.
  Es el encargado de administrar todos los recursos y hacer el seguimiento de la planificación verificando que todo salga bien.

- **Equipo del proyecto**: Grupo de personas comprometidas en alcanzar un conjunto de objetivos los cuales son responsables, se complementan (intercambian conocimientos y habilidades) para trabajar en equipo y lograrlo.

Planificacion

1. **Definición del alcance:**

ALCANCE DEL PROYECTO	ALCANCE DEL PRODUCTO
Todo trabajo que debe hacerse para entregar el producto con todas las características y funciones especificadas en el objetivo	Todas las características que pueden incluirse en un producto o servicio

2. **Definición del proceso y ciclo de vida:**
   El proceso de desarrollo es el conjunto de actividades que necesito ejecutar para construir el producto de software y el ciclo de vida es de qué manera ejecutar esas actividades.
3. **Estimacion:**

Estimar todas las características necesarias para la planificación con cierto nivel de incertidumbre, se definen las siguientes características.
    - Tamano
    - Esfuerzo
    - Calendario
    - Costo
    - Recursos criticos
4. **Gestion de riesgos:**
   Identificar todos los riesgos probables de ocurrir o los más impactantes. Calcular la exposición de riesgo (multiplicando las variables de impacto y ocurrencia), con esto determinamos los riesgos principales y cuales voy a gestionar para disminuir el impacto.
5. **Asignación de recursos**
6. **Programación de proyectos**
7. **Definición de métricas:**

DE PROCESO	Mismas métricas del proyecto pero despersonalizadas de uno en particular. Nos deja saber si como organización estamos trabajando bien o mal. Ej: Porcentaje de proyectos que terminan en término y porcentaje de proyectos que terminan fuera de término
DE PROYECTO	Nos permiten saber si un proyecto de software en ejecución se está cumpliendo de acuerdo a lo planificado o no Ej: Comparación del tiempo calendario real con el planificado
DE PRODUCTO	Miden características que tienen una relación directa con el producto que estamos construyendo Ej: Métricas de defectos, todas las métricas de tamaño

8. **Monitoreo y control:**
   El PM determina si se cumple o no lo establecido en el plan de proyecto.

Gestión de productos

Se centra en la definición, planificación y evolución de un artefacto informático construido para satisfacer una necesidad específica basándose en requerimientos concretos.

Tiene como objetivo priorizar la creación de valor para el cliente y evitar la inversión de esfuerzo en elementos considerados como desperdicio.

Pilares de la gestión de un producto de software

- **Enfoque en el valor real:** Casi el 80% de las características de un software no se usan, y solo el 7% siempre son usadas. El desafío de la gestión de producto es encontrar y enfocarse únicamente en ese 7% que aporta valor.

- **Evolución del producto:** Se inicia con una base **funcional**. Luego, el gestor, debe guiar la evolución para que sea **confiable**, **usable**, **conveniente** y placentero **y significativo** para el usuario.

- **Propósito claro:** Los productos se gestionan y desarrollan para alcanzar objetivos específicos.

- **Planificación ágil:** La visión integral del producto se centraliza y materializa en el *Product Backlog*. Esta luego se descompone en entregas frecuentes (*releases*) y se van construyendo mediante iteraciones sucesivas.

- **Productividad redefinida:** Asegurarse que se está construyendo lo correcto todos los días.

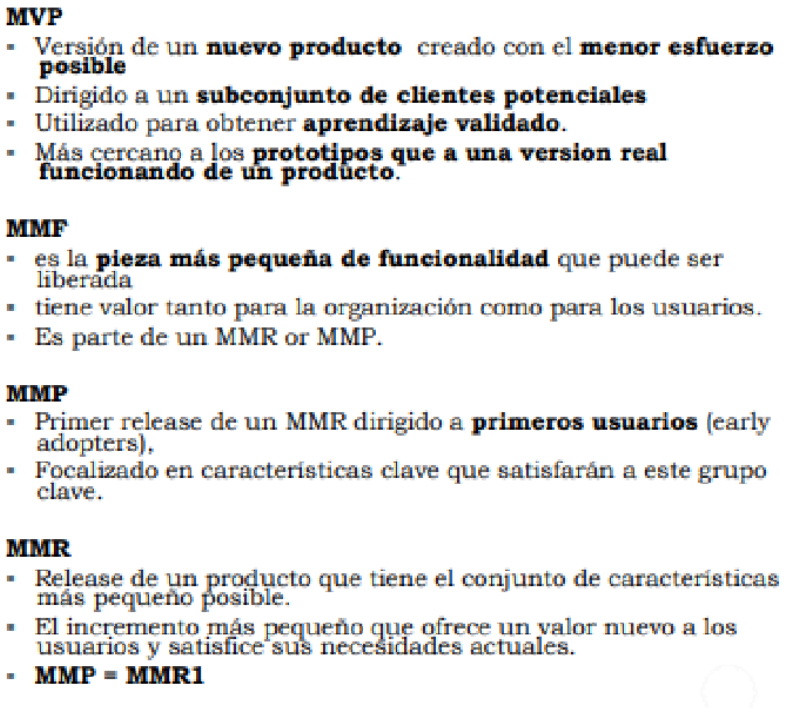
- **Uso de herramientas de validación temprana:** Definir hipótesis y validarlas a través de un MVP (Producto Mínimo Viable) para recopilar aprendizaje sobre los clientes con el menor esfuerzo. También se emplean metodologías como Design Thinking (empatía y necesidades del usuario) y Lean UX (ciclo construir-medir-aprender).

MVP - Mínimo Producto Viable

Concepto de Lean Startup que se enfoca en el impacto del aprendizaje en el desarrollo de nuevos productos.

Es una versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre los clientes con el menor esfuerzo posible.

- Se basa en la idea de que producir un producto real para observar el comportamiento real de la gente es mucho más confiable que preguntarles qué harían.
- El producto, para que sea un MVP, debe tener el valor suficiente para que las personas estén dispuestas a usarlo o comprarlo, debe tener beneficios futuros para los usuarios y proporcionar un ciclo de retroalimentación que guíe el desarrollo futuro.
- Permite probar 2 hipótesis:
    - Hipótesis de valor: Si el producto realmente entrega valor al cliente.
    - Hipótesis de crecimiento: Como descubrirán el producto nuevos clientes



Product Backlog

Artefacto fundamental en Scrum y Agile

Es una lista emergente, ordenada y priorizada de características y requerimientos (ítems o PBIs) que se necesitan para mejorar el producto.

El *PO* es el encargado de definir y priorizar la lista, definiendo qué requerimientos se necesitan primero según el valor de negocio que aportan.

El Product Backlog nunca está definido al 100%, ya que a medida que el proyecto transcurre, los ítems pueden borrarse, agregarse o ser re-priorizados.

Design Thinking

Metodologia iterativa para crear servicios cetrados en el usuario, tiene cinco etapas obligatorias:

- Empatizar
- Definir
- Idear
- Prototipar
- Probar

Lean UX

Enfoque sustentado en el circuito **Construir - Medir - Aprender**, tiene como objetivo permitir equivocarse en estadios precoces cuando es barato reconducir la idea

Requerimientos Ágiles

Está centrada en construir valor de negocio y satisfacer al cliente mediante la entrega temprana y continua de software útil. El agilismo asume que la incertidumbre es inevitable y que los requerimientos irán madurando y descubriendo a medida de que el proyecto avanza.

Fundamentos

- **Enfoque en el valor:** Construir el producto que resuelva un problema y aporte valor al negocio del cliente, no únicamente cumplir un plan o entregar documentos.

- **Solo lo suficiente (Just in Time):** Los requerimientos se detallan y analizan únicamente cuando se necesitan. Se empieza con una visión general y se profundiza iterativamente para generar una retroalimentación rápida.

- **Evitar el BRUF (Big Requirements Up Front):** Las metodologías ágiles rechazan la práctica de especificar el 100% de los requerimientos al inicio del proyecto, porque esto genera desperdicio frente a la alta probabilidad de que las necesidades del usuario cambien.

Triple Restriccion Agil

- **Alcance Variable:** El tiempo y los recursos son fijos, mientras que el alcance es variable.
- **Compromiso de Entrega:** Se evalúa, en función del tiempo y de los recursos disponibles, qué porción de funcionalidades de mayor valor puede entregarle al cliente en cada ciclo.

Otros fundamentos (Comunicacion, tipos, gestión del cambio)

- **Cara a Cara sobre la documentación:** El medio de comunicación más eficiente es la conversación directa entre técnicos y no técnicos (que trabajarán en conjunto durante todo el proyecto). La documentación formal (ERS) se reemplaza por la disponibilidad del cliente y las User Stories.

- **Dominio del Problema:** Nos interesa trabajar los *requerimientos de Negocio* y los *requerimientos de Usuario*, dejando los requerimientos funcionales, no funcionales y de implementación como tareas de transformación para el equipo.

- **Abiertos al Cambio:** "*El usuario dice lo que quiere cuando recibe lo que pidió*", por eso se aceptan cambios en los requerimientos, incluso en etapas finales del desarrollo, usando este factor como una ventaja competitiva.

User Story

Tecnica para trabajar requerimientos de usuario en entornos agiles.
Es una descripción corta de una funcionalidad esperada del producto que aporta valor a un usuario del sistema.
Funciona como recordatorio para el equipo de desarrollo y el PO de que deben conversar en el futuro sobre una necesidad de negocio.

Enfoque

- **Porciones verticales:** Las US se consideran porciones verticales del producto, porque para entregar una característica terminada, deben abarcar todas las capas de la arquitectura (UX, lógica de negocio y DB)

- **Multipropósito:** Representan una necesidad del usuario, son una descripción del producto, sirven como ítem de planificación y actúan como un mecanismo para diferir una conversación detallada hasta el momento adecuado.

Las "3 C"

- **Tarjeta (Card):** Es la parte visible y escrita que contiene la descripción de la historia y se usa para planificar.

- **Conversación (Conversation):** Parte más importante y no visible, es el diálogo directo entre el PO y el equipo de desarrollo, donde se negocian y desarrollan los detalles específicos para implementar la US.

- **Confirmación (Confirmation):** Son las pruebas de aceptación y condiciones que determinan cuando la historia está completa y cumple lo esperado por el PO.

Formato

Se escriben como:
“Como <ROL> yo puedo <ACTIVIDAD> de tal forma que <VALOR DE NEGOCIO QUE RECIBO>”

El **ROL** representa a quien hará la acción o recibirá el beneficio.
La **ACTIVIDAD** es la acción del sistema.
El **VALOR DEL NEGOCIO** es la justificación de por que esa acción es necesaria y permite priorizarla.

Criterios y pruebas de aceptación

- **Criterios de aceptación:** Definen los límites de la historia y ayuda a tener una visión de los requerimientos funcionales mínimos. Son redactados a altos niveles, indicando una intención.

- **Pruebas de aceptación:** Expresan detalles resultantes de las conversaciones. Se prueban escenarios exitosos y fallidos, y el PO acepta la US solo si se cumplen todas esas pruebas.

Filtros de calidad

- **Definition of Ready:** Medida de calidad que determina si la US está lo suficientemente bien formulada como para ingresar a una iteración de desarrollo.

- **Modelo INVEST:** Estándar de calidad mínimo para que la US cumpla con el DoR. Dice que la historia debe ser: **I**ndependiente, **N**egociable, **V**aluable, **E**stimable, **S**mall y **T**esteable.

- **Definition of Done:** Check List que establece cuando la US fue implementada correcta y completamente para ser mostrada al PO y pasar a producción.

Niveles de Abstracción y Casos Especiales

- **Epica:** US de granularidad gruesa, demasiado grande para entrar en una iteración, debe ser dividida en US mas pequeñas para implementarse.

- **Tema:** Conjunto de US agrupadas porque están relacionadas, se tratan como una entidad simple a la hora de estimar o planificar.

- **Spike:** US que se genera cuando existe una incertidumbre técnica o funcional que impide estimar su tamaño. Son una excepción a la regla, necesitan ser investigadas o prototipadas, y se usan para reducir riesgos antes de desarrollarlas.

Estimaciones Agiles

Aproximaciones usadas para predecir la completitud de un trabajo y gestionar los riesgos del proyecto, asumiendo de que la **incertidumbre es inevitable**.
Las estimaciones no son planes, promesas ni compromisos formales en AGILE.

Diferencias con el Enfoque Tradicional

- **Medidas Relativas vs Absolutas:** En lugar de intentar predecir con precisión absoluta (costoso y propenso a errores), el agilismo estima mediante la **comparación** relativa de tamaños, ya que las personas no son buenas estimando en términos absolutos, pero si comprando magnitudes.

- **Quien Estima:** En el enfoque tradicional estima un gerente o líder, en el ágil, *estima quien hace el trabajo*. El equipo de desarrollo realiza la estimación de forma colectiva y colaborativa. El PO participa aclarando dudas, pero no estima.

- **Ajuste por Alcance:** Como en las metodologías ágiles el tiempo y los recursos son fijos, mientras el alcance es variable, una falla en la estimación no implica extender el tiempo del proyecto, sino que se ajusta el alcance entregado

Unidad de medida → Story Points

- Es la unidad de medida que se usa para medir el tamaño relativo de una US frente a otra.

- No mide tiempo, sino que combina tres variables:
    - *Incertidumbre:* cuanta información falta sobre el negocio.
    - *Complejidad:* cantidad de partes y relaciones que tiene la funcionalidad.
    - *Esfuerzo:* horas de trabajos ideales o lineales requeridas.

- El tamaño depende de la duración o el esfuerzo absoluto, ya que el esfuerzo real puede variar dependiendo de las habilidades y la experiencia de la persona que realice la tarea.

Metrica de progreso: Velocity

- Métrica principal de progreso del equipo, no se estima, sino que se **calcula** o **mide**.
- Cálculo: Suma de SP asignados a las US que el equipo completo al 100% y fueron aceptadas al PO al finalizar un Sprint.

- Únicamente se cuentan los SP finalizados, no los que no lo están o estaban parcialmente.
- Nos permite predecir con cuánto trabajo se pueden comprometer en futuras iteraciones y ayuda a derivar la duración total del proyecto.

Poker Planning | Técnica de estimación

- Técnica colaborativa basada en cartas que combina el juicio experto, la analogía y la técnica Delphi, promoviendo que el equipo debate y llegue a un consenso sobre el tamaño de cada historia.
- **Historia canónica:** Se define primero una historia de referencia simple (baja incertidumbre) y se le asigna un puntaje base (1, 2 SP) para comparar todas las demás, con ella.
- **Escala de Fibonacci:** Para los puntajes se usa la serie de Fibonacci (1, 2, 3, 5, 8, 13, etc.), porque la complejidad e incertidumbre del software suele incrementarse en forma exponencial.
- **Limites:** Si a una historia se le asigna un puntaje muy alto (13 o más), significa que estamos ante una US **Épica** para entrar en un solo sprint, y deberá ser dividida en partes más pequeñas.

Reestimacion

Solo se aconseja estimar una US si la comprensión del equipo cambió al obtener más información.
No se debe re-estimar si el equipo avanza más lento de lo esperado ya que las inexactitudes en los tiempos se equilibran solas con la métrica de velocidad.

Framework Scrum

Scrum: framework ligero, diseñado para ayudar a personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos.

- No proporciona instrucciones detalladas, proporciona reglas que guían las interacciones basándose en el **empirismo** y el **pensamiento Lean**.

Pilares de SCRUM

- **Transparencia:** El proceso y el trabajo emergente debe ser completamente visible para los que realizan el trabajo y los que lo reciben, evitando decisiones basadas en información oculta.

- **Inspección:** Los artefactos y el progreso hacia los objetivos deben revisarse frecuentemente para detectar variaciones indeseadas.

- **Adaptación:** Si se detectan desviaciones fuera de los límites aceptables, el proceso debe reajustarse lo antes posible.

- **Valores de Scrum:** Para que estos pilares funcione, el equipo debe interiorizar cinco valores:
    - Compromiso
    - Enfoque
    - Apertura
    - Respeto
    - Coraje

Roles del SCRUM

Un equipo de scrum es una unidad cohesionada, multifuncional y autogestionada. No hay subequipos ni jerarquías, son de 10 o menos personas.

- **Product Owner:** Responsable de maximizar el valor del producto y gestionar el Product Backlog. Representa las necesidades de los stakeholders o del negocio.

- **Scrum Master:** Es un líder del equipo y organización, asegura que el Scrum se entienda y se aplique bien, promueve la autogestión, facilita eventos y elimina impedimentos que bloqueen al equipo.

- **Desarrolladores:** Encargados de crear el incremento útil y de valor de cada Sprint, son los responsables de crear el Sprint Backlog, asegurar la calidad y adaptar el plan diariamente.

Eventos y Timebox

Todo evento Scrum está limitado por un Timebox, para obligar al equipo a no perder tiempo y a negociar el alcance en vez de extender las fechas.

- **Sprint:** Evento contenedor de todas las ceremonias, funcionando como un proyecto corto con duración fija (1 mes o menos).

- **Sprint Planning:** Inicia el Sprint definiendo el trabajo a realizar, responde a tres preguntas: (Max. 8hs para un sprint de 1 mes)
    - ¿Por qué es valioso este Sprint? → Objetivo
    - ¿Qué se puede hacer? → Ítems seleccionados
    - ¿Cómo se realizará? → Plan de acción

- **Daily Scrum:** Reunión corta (15m), misma hora y lugar, exclusiva para desarrolladores. Sirve para inspeccionar el objetivo y adaptar el trabajo para el día siguiente.

- **Sprint Review:** Sesión de trabajo donde el equipo presenta los resultados del Sprint a los stakeholders. Sirve para inspeccionar lo logrado, recibir feedback y adaptar el Product Backlog para el futuro (Max. 4hs para un sprint de 1 mes)

- **Sprint Retrospective:** Cierre del Sprint. Reunión interna del equipo para ver cómo trabajaron en términos de individuos, procesos y herramientas, con el fin de mejorar para el próximo ciclo. (Max. 3hs para un sprint de 1 mes)

Artefactos

Representan trabajo o valor y están diseñados para maximizar la transparencia.

- **Product Backlog:** Lista emergente y priorizada de todo lo necesario para mejorar el producto:
    - *Compromiso → Objetivo del producto: Describe el estado futuro del producto a largo plazo que se quiere alcanzar.*

- **Sprint Backlog:** Plan en tiempo real creado por y para los desarrolladores, compuestos por los ítems seleccionados para la iteración actual y el plan de acción.
    - *Compromiso → Objetivo del Sprint*

- **Incrementó:** Un paso concreto y utilizable que se suma a los incrementos anteriores para acercarse al Objetivo del Producto
    - *Compromiso → Definition of Done (DoD)*

SCM | Gestión de Configuración de Software

Disciplina de soporte transversal a todo el proyecto.
Su objetivo es establecer y mantener la integridad del producto a lo largo de todo su ciclo de vida.
Surge como medida preventiva para resolver graves problemas de desarrollo (pérdida de componentes, de cambios, superposición de modificaciones, fallas de regresión, doble mantenimiento).

Conceptos

- **Ítem de Configuración (IC):** Todo artefacto que forma parte del producto o proyecto, susceptibles a sufrir cambios, y que deben ser almacenados en un repositorio para conocer su estado.
    - Ejemplos: Codigo fuente, arquitectura, casos de prueba, manuales de usuario.
    - Se clasifican en *ítems de producto, de proyecto o de iteración.*

- **Versión:** Instancia particular de un IC que difiere de otras en un momento específico del tiempo.

- **Variante:** Versión de un IC que evoluciona de manera separada para representar configuraciones alternativas (Ej. adaptar el mismo software para otro SO).

- **Configuración:** Conjunto de todos los IC con su versión específica, en un momento dado, funciona como una foto del sistema.

- **Repositorio:** Contenedor (centralizado en un servidor o descentralizado en múltiples clientes) que mantiene la historia de cada ítem, sus atributos y relaciones.

- **Línea Base:** Conjunto de IC que fueron construidos, revisados y aprobados. Sirve como un punto de referencia estable para demostrar madurez y usarse como base para otros desarrollos. Existen líneas base de *especificación* (documentos, requerimientos) y *operacionales* (código ejecutable que paso por controles de calidad)

- **Rama y Merge:** Las ramas permiten bifurcar el desarrollo de un software para experimentar o resolver de forma paralela. El merge ocurre cuando se fusionan esas configuraciones hacia una rama principal estable.

Actividades

- **Identificación de Ítems:** Asigna un ID a cada artefacto, definiendo convenciones y reglas de nombrado, estructura y ubicación dentro del repositorio.

- **Control de Cambios:** Procedimiento formal para alterar una Baseline. Dirigido por el *Comité de Control de Cambios*, que recibe la propuesta, la analiza (impacto técnico y de recursos) y la autoriza o rechaza. También notifica a los interesados.

- **Auditorías de Configuración:** Controles objetivos realizados por un auditor externo al equipo de desarrollo, valida la integridad y calidad. La *auditoría física* verifica que los IC del repositorio coincidan con el plan y el nombrado. La *auditoría funcional* evalúa la trazabilidad.

- **Informes de Estado:** Dan un registro automatizado sobre la evolución del sistema y las modificaciones de las baseline, asegurando que todos los involucrados trabajen en la última versión.

SCM en Ambientes Agiles

SCM en Agile, cambia su enfoque, busca hacer un seguimiento y coordinar el desarrollo. Al ser equipos auto-organizados, se evitan las auditorías externas y el comité formal de cambios en el Sprint, priorizando la adaptación constante y la automatización intensiva.

- **Integración Continua (CI):** Práctica donde los desarrolladores integran su código a un repositorio. El código se verifica automáticamente mediante pruebas unitarias para detectar problemas rápidamente.

- **Entrega Continua (CD):** Garantiza que el software se construya de tal manera que siempre está liberado a producción, incluye automatización de pruebas de aceptación, requiere intervención humana para realizar el pase final.

- **Despliegue Continuo:** El software modificado y aprobado pasa al entorno de producción sin ninguna intervención humana, usando *pipelines* (flujos de trabajo) automatizados.

- **Estrategias de Despliegue:** Se usan técnicas como *Canary Deployment* (libera la actualización a una porción de usuarios para probar su estabilidad antes de extenderla a todos) o *Blue-Green Deployment* (Se transfiere el tráfico gradualmente a un entorno azul, con la version vieja, a un entorno verde idéntico con la version nueva)

Framework para escalar SCRUM / Nexus

La necesidad de escalar *Scrum* surge porque se impone un límite en la cantidad máxima de integrantes del equipo, por lo que para gestionar productos de mayor envergadura y complejidad es necesario utilizar un framework como ***Nexus***

Nexus

Son roles, eventos, artefactos y técnicas que vinculan el trabajo de aproximadamente 3 a 9 equipos Scrum.
Estos equipos trabajan sobre un **único Product Backlog común** para construir un incremento integrado de producto “Terminado”, guiados por un **solo Product Owner**.
Su propósito es lidiar con las dependencias entre equipos, las cuales se dan por superposición de requerimientos o por la necesidad de mapear el conocimiento del dominio para minimizar interrupciones.

Roles

- **Nexus Integration Team (NIT):** Responsable de asegurar que se produzca un incremento integrado al menos una vez en cada Sprint, abordando las restricciones técnicas y no técnicas que impiden la entrega. Sus miembros suelen provenir de los mismos Scrums Teams para ayudar en la adopción de herramientas y prácticas de integración.

- **Product Owner:** Maximiza el valor del producto, es el único responsable de la gestión eficaz del Product Backlog general.

- **Scrum Master:** Asegura que el marco Nexus se entienda y aplique, pudiendo desempeñar este rol simultáneamente en uno o más de los Scrums Teams que conforman el Nexus.

Eventos

Se añaden o extienden los eventos tradicionales de Scrums, incluyendo un bloque de tiempo adicional a las ceremonias correspondientes

- **Refinamiento entre equipos:** Actividad continua donde el *PB* se descompone para hacer que las dependencias sean transparentes, se identifiquen y se minimicen, ayudando a prever que equipo entregará cada elemento.

- **Nexus Sprint Planning:** Representantes de cada equipo y el PO coordinan las actividades para el Sprint.
    - Se obtiene un *Objetivo de Sprint (para cada equipo), un Objetivo de Nexus, un Nexus Sprint Backlog y los Sprint Backlogs individuales.*

- **Nexus Daily Scrum:** Solo asisten los representantes de cada equipo para discutir problemas de integración e inspeccionar el progreso hacia el objetivo del Nexus. Esta reunión complementa a la Daily Scrum individual de cada equipo, la cual se centra posteriormente en crear planes para abordar los problemas de integración planteados.

- **Nexus Sprint Review:** Reemplaza a las *Sprint Reviews* de cada equipo individual. Se realiza al final del Sprint para brindar retroalimentación sobre el incremento integrado que todo el Nexus construyó de forma conjunta.

- **Nexus Sprint Retrospective:** Inspecciona como fue el Sprint a nivel global para planificar mejoras. Se nutre de las retrospectivas de cada equipo, usando inteligencia de abajo hacia arriba para centrarse en los problemas sistémicos.

Artefactos y Compromisos

- **Product Backlog:** Hay solo 1 para todo el Nexus.
    - Compromiso → Objetivo del Producto (meta a largo plazo)

- **Nexus Sprint Backlog:** Compuesto por el Objetivo del Sprint de Nexus y los elementos del PB seleccionados por cada equipo, resaltando las dependencias durante el Sprint.
    - Compromiso → Objetivo del Sprint de Nexus

- **Integrated Increment:** Suma actual de todo el trabajo integrado completado por el Nexus.
    - Compromiso → Definición de Hecho (DoD) ← Es responsabilidad del NIT que sea aplicable, todo equipo deben adherirse a ella

LeSS (Large - Scale Scrum)

- Se aplica al desarrollo de productos donde intervienen entre 2 a 8 equipos conjuntos.
- Principio fundamental → *“Scrum a gran escala es Scrum”*
- **1 solo PO**
- **1 unico PB**
- **1 solo Sprint Comun**

Roles

- **Product Owner:** Encarga de gestionar y priorizar el Product Backlog
- **Scrum Master:** Tiene un enfoque sistémico orientado a la organización, sirve a varios equipos en simultáneo (1 a 3).
- **Equipos:** Autogestionados y multifuncionales. Cada equipo coordina e integra su trabajo con los demás, manejan sus relaciones.

Eventos

- **Sprint Planning One (Que Hacer):** Participa el PO y los equipos (o representantes), seleccionan qué ítems del Backlog trabaja cada equipo e identifican oportunidades de colaboración.

- **Sprint Planning Two (Como hacerlo):** Cada equipo, por separado, decide cómo se implementarán los ítems seleccionados y crean su propio Sprint Backlog.

- **Daily Scrum:** Reunion propia de cada equipo de forma independiete

- **Product Backlog Refinement:** Realizado por todos los equipos juntos, se dividen elementos grandes, se estima y se aclara el trabajo futuro.

- **Sprint Review:** Hay ***una sola*** revisión común para todo el producto. Los equipos, el PO, los clientes y los stakeholders se reúnen para inspeccionar el incremento global integrado,

- **Retrospectivas:**
    - **Sprint Retrospective:** Propia de cada equipo
    - **Overall Retrospective:** General, se discuten problemas organizacionales, sistémicos y de coordinación entre los equipos.

Artefactos

- **Product Backlog:**
- **Sprint Backlog:**
- **Incremento:** La salida de cada Sprint es solo un incremento ENTREGABLE, integra el trabajo de todos los equipos y debe complir con DoD.

LeSS Huge

Si hay más de 8 equipos.
Se mantiene el Sprint común y el único incremento, se agrega un *Area Product Manager,* varios *Area Product Owners* y el backlog se divide en *Area Product Backlogs*

Filosofía Lean - Método Kanban en el contexto del desarrollo de software

Filosofia Lean - (La Estrategia)

Nacida en la manufactura de Toyota y adaptada al software, su obsesión absoluta es **satisfacer al cliente eliminando todo lo que no agregue valor.**

Principios

1. **Eliminar el desperdicio (Muda):** *Pilar central.* En software, el desperdicio no es material físico, sino tiempo y esfuerzo en cosas que el cliente no usará.
   Los más típicos son:
    - *Características extra (Goldplating):* Programar cosas que están fuera del 20% del valor.

    - *Trabajo a medias:* Código sin testear o requerimientos a medio definir.
    - *Esperas:* Un desarrollador inactivo porque el servidor no está listo.
    - *Cambios de tareas (Multitasking):* Perder el foco saltando entre proyectos.
        - "Deja de empezar y comienza a terminar"
    - *Defectos:* Errores que obligan a retrabajar.

2. **Amplificar el aprendizaje:** Fomentar una cultura de mejora continua en el equipo (autoorganizado) donde el conocimiento se comparte en toda la organización, evitando definir todo el diseño prematuramente.

3. **Embeber la integridad conceptual:** Pensar en la arquitectura del producto desde el principio para que sea escalable, robusto y consistente (RNF), en lugar de probar la calidad final.

4. **Diferir compromisos (El último momento responsable):** No tomar decisiones irreversibles hasta que sea estrictamente necesario. Mantener las opciones abiertas reduce el riesgo ante la incertidumbre.

5. **Dar poder al equipo:** Delegar las decisiones técnicas y las responsabilidades a las personas que realmente hacen el trabajo, entrenando líderes en lugar de dar micro-órdenes.

6. **Ver el todo:** Tener una visión holística (de conjunto) del producto y del negocio, evitando optimizar solo una parte del código si el sistema global va a fallar.

7. **Entrega rápida:** Acortar los ciclos de desarrollo para entregar valor rápidamente al mercado, obtener feedback y evitar que lo construido se vuelva obsoleto.

Kanban - (La Ejecución)

NO es una metodología de gestión de proyectos → es un enfoque para la Gestión del Cambio. Sirve para gestionar el flujo de requerimientos (tickets) de manera continua, ideal para equipos de soporte o mantenimiento donde Scrum no sirve.

Funciona como un sistema **Pull** (el desarrollador "tira" de la tarea cuando tiene capacidad) y no **Push** (el jefe le "empuja" tareas al desarrollador).

Principios

1. **Visualizar el flujo de trabajo:** Mapear físicamente o digitalmente el proceso (*To Do, In Progress, Testing, Done*). Todo el trabajo oculto debe volverse visible mediante tarjetas.

2. **Limitar el Trabajo en Curso (WIP - Work In Progress):** Es el corazón de Kanban. Se pone un límite matemático a las columnas. Si se llega al límite, el equipo debe dejar de tomar tareas nuevas y ayudar a terminar las estancadas para que el flujo no se ahogue.

3. **Gestionar el flujo:** Identificar cuellos de botella (*Ej. Si la columna de Testing está siempre al límite del WIP, hay que mover programadores para que ayuden a testear).*

4. **Hacer explícitas las políticas de calidad:** Definir claramente los criterios exactos que debe cumplir una tarjeta para poder pasar de una columna a la siguiente

5. **Mejorar colaborativamente:** Evolucionar gradualmente el proceso mediante el aprendizaje del equipo.
6. **Respetar los roles actuales:** A diferencia de Scrum, Kanban no te obliga a crear los roles de “Scrum Master” o “Product Owner”. Trabajas con la estructura que ya tienes.

Clases de Servicio en Kanban

Para priorizar el trabajo que fluye por el tablero, Kanban usa “Clases de Servicio” (*no es una cola FIFO simple*).

- **Expreso:** Tareas críticas. Se deben resolver inmediatamente. Son las únicas que tienen permiso para romper el límite WIP.

- **Fecha fija:** Requerimientos que tienen un límite de tiempo legal o comercial inamovible.

- **Estándar:** Funcionalidades normales. Siguen el flujo FIFO y deben respetar el WIP.

- **Intangible:** Tareas técnicas bajo riesgo (refactorización)

Metricas

Al ser de flujo continuo, no se miden en “SP”, sino por **Tiempo**.

- **Lead Time (Vista del Cliente):** El tiempo total que transcurre desde que el cliente hace un pedido, o sea entra a PB, hasta que la funcionalidad está terminada y entregada.

- **Cycle Time (Vista Interna):** El tiempo real de trabajo. Cuenta desde que el equipo toma la tarjeta y empieza a programarla, hasta que se termina.

- **Touch Time:** Tiempo neto que el equipo estuvo tocando el código (descuenta tiempos muertos)

Métricas en los 3 enfoques| Tradicional | Lean | Agile

TRADICIONAL	AGILE	LEAN
• Esfuerzo • Tiempo • Costos • Riesgos	• Velocidad • Capacidad • Running Tested Features (RTF)	• Lead Time • Cycle Time • Touch Time • Eficiencia del Proceso

En el Enfoque Tradicional (Procesos Definidos)

Medición amplia, queda a criterio del Líder del Proyecto, se basan en medir el cumplimento de un plan inicial y se dividen en tres grandes dominios:

- **Métricas de Producto:** Miden el software en sí mismo *(cantidad lineas de código, defectos por severidad).*

- **Métricas de Proyecto:** Son de interés para el Líder de Proyecto para evaluar el estado actual *(tamaño, esfuerzo en horas lineales, tiempo y defectos)*. Su utilidad termina cuando el proyecto finaliza.

- **Metricas de Proceso:** Despersonalizan las métricas de proyecto para medir el rendimiento de toda la organización a largo plazo *(desviación promedio del calendario en todos los proyectos de la empresa).*

- **Métricas clave → Esfuerzo, Tiempo, Costos y Riesgos**

En el Enfoque Ágil (Procesos Empíricos | Scrum)

El agilismo es minimalista y se rige por el principio de que *el software funcionando es la principal medida de progreso.*

- **Velocidad:** Mide la cantidad de trabajo (SP) que el equipo logró completar al 100% y que el PO aprobó en la Sprint Review. Tiende a estabilizarse con el tiempo.

- **Capacidad:** Estimación de cuánto trabajo (SP o hs ideales) va a poder completar y comprometer el equipo para el próximo Sprint.

- **Running Tested Features (RFT):** Mide la cantidad de características del software que han sido implementadas, testeadas con éxito y están funcionando. Como limitación, solo cuenta la cantidad pero no la complejidad de cada una.

En el Enfoque Lean (Kanban)

Enfocado exclusivamente en la entrega rápida, el flujo continuo y la eficiencia del tiempo, no en SP ni estimaciones de esfuerzo

- **Lead Time (Vista del Cliente):** El tiempo total que transcurre desde que el cliente hace un pedido, o sea entra a PB, hasta que la funcionalidad está terminada y entregada. Mide el “Ritmo de Terminación”.

- **Cycle Time (Vista Interna):** El tiempo real de trabajo. Cuenta desde que el equipo toma la tarjeta y empieza a programarla, hasta que se termina. Mide el “Ritmo de Entrega”.

- **Touch Time:** Tiempo real y neto en el que un ítem fue trabajado por el equipo, descontando todo el tiempo que estuvo inactivo en columnas de espera o bloqueado.

- **Eficiencia del Ciclo de Proceso:** Se calcula dividiendo el Touch Time sobre el Lead Time. Mientras más cercano a 1, más eficiente el proceso.

Practicas Continuas

SCM en Agile, cambia su enfoque, busca hacer un seguimiento y coordinar el desarrollo. Al ser equipos auto-organizados, se evitan las auditorías externas y el comité formal de cambios en el Sprint, priorizando la adaptación constante y la automatización intensiva.

Su propósito es automatizar al máximo el proceso desde que el programador escribe una línea de código hasta que esa funcionalidad llega al usuario final, asegurando calidad, estabilidad y velocidad.

- **Integración Continua (CI):** Práctica donde los desarrolladores integran su código a un repositorio. El código se verifica automáticamente mediante pruebas unitarias para detectar problemas rápidamente. (Pruebas unitarias → TDD (Test Driven Development)

- **Entrega Continua (CD):** Garantiza que el software se construya de tal manera que siempre pueda ser liberado a producción en cualquier momento, incluye automatización de pruebas de aceptación, requiere intervención humana para realizar el pase final.

- **Despliegue Continuo:** El software modificado y aprobado pasa al entorno de producción sin ninguna intervención humana, usando *pipelines* (flujos de trabajo) automatizados.

- **Estrategias de Despliegue:** Se usan técnicas como *Canary Deployment* (libera la actualización a una porción de usuarios para probar su estabilidad antes de extenderla a todos) o *Blue-Green Deployment* (Se transfiere el tráfico gradualmente a un entorno azul, con la version vieja, a un entorno verde idéntico con la version nueva)

Aseguramiento de Calidad de Proceso y de Producto

Conceptos de Calidad

Definicion Formal

La calidad se define como el cumplimiento de los requerimientos funcionales y de performance explícitamente definidos, de los estándares de desarrollo explícitamente documentados y de las características implícitas esperadas del desarrollo de software profesional.

Definición Orientada a Necesidades

Todo aspecto y característica de un producto o servicio que se relacionan con su habilidad para alcanzar las necesidades manifiestas o implícitas del cliente.

Subjetividad y Contexto

La calidad está relacionada directamente con la persona (*es subjetiva a quien la analice*), las circunstancias, el contexto y la edad en un momento de tiempo dado. Lo que para una persona representa calidad, puede no serlo para otra, ya que se vincula directamente con sus expectativas y necesidades particulares.

Perspectivas de Evaluación

El software y su calidad pueden analizarse desde múltiples perspectivas, fundamentalmente como proceso y como producto, siendo la calidad un factor difícil de medir en forma absoluta.

Ventajas y Desventajas de la Calidad

Hoy en día, trabajar con calidad es fundamental para la producción de software, ya que el proceso es la base o fundamento principal para mejorar la calidad resultante.

Ventajas

- **Ventaja Competitiva:** Aspecto competitivo indispensable que permite sobrevivir y competir en el mercado internacional, a menudo mediante la certificación de estándares de calidad de proceso.

- **Retención y Beneficios:** Retiene a los clientes e incrementa los beneficios económicos, ya que la calidad se traduce en el cumplimiento de las expectativas del cliente en todos los sentidos.

- **Equilibrio Costo-Efectividad:** Trabajar con calidad determina un equilibrio positivo entre costo y efectividad. Está comprobado que trabajar con calidad es más barato, ya que previene defectos y reduce el retrabajo y los costos asociados a corregir fallas tardías.

- **Satisfacción Integral:** Un software de calidad ofrece el cumplimiento de las expectativas del cliente, de los usuarios, y satisface las necesidades tanto de la gerencia como del propio equipo de desarrollo y mantenimiento.

Desventajas y Problemas de la NO Calidad

La falta de aseguramiento de calidad, o una calidad reducida, se evidencia a través de ciertas problemáticas que afectan directamente a la percepción del cliente y el éxito del proyecto.

- **Desvíos en el Proyecto:** Genera retrasos en las entregas, costos excedidos respecto a los presupuestados y trabajo fuera de hora para el equipo.

- **Fallas en el Producto:** Resulta en un software que no hace lo que debería hacer o lo que se pidió, acompañado frecuentemente de requerimientos no claros y la falta de aplicación de Gestión de Configuración de Software (SCM).

- **Incumplimiento de Compromisos:** Provoca la falta de cumplimiento de los compromisos asumidos y la aparición del fenómeno del 90-90 (donde se percibe que el 90% está hecho, pero resta un 90% del esfuerzo real).

Aseguramiento de Calidad

Definición de procesos y estándares que deben conducir a la obtención de productos de alta calidad. Es actuar como medida preventiva, a diferencia del testing que llega tarde.

Actividades para Asegurar la Calidad

Se divide la gestión en tres actividades fundamentales

- **Aseguramiento de Calidad:** Consiste en establecer los estándares y procedimientos de calidad. Aquí se elige contra qué estándares se van a efectuar las comparaciones (revisiones técnicas y auditorías). En la práctica, es insertarse en el proceso de desarrollo mientras el software se está construyendo.

- **Planificación de Calidad:** Se seleccionan los procedimientos y estándares de calidad generales y se los adapta para un proyecto en particular.

- **Control de Calidad:** Se ejecuta el control propiamente dicho de acuerdos a los procedimientos y estándares de calidad elegidos en la planificación.

Funciones y Prácticas del Aseguramiento de Calidad

Las tareas operativas específicas que deben llevar a cabo el área de calidad son:

- Desarrollo de herramientas adecuadas, técnicas, métodos y estándares que están disponibles para realizar las revisiones de Aseguramiento de Calidad.
- Adaptación de los procedimientos de Aseguramiento de Calidad para cada proyecto.
- Evaluación de la planificación del Proyecto de Software.
- Evaluación de los procesos de planificación y control de proyectos.
- Evaluación de Requerimientos.
- Evaluación del Proceso de Diseño.
- Evaluación de las prácticas de programación.
- Evaluación del proceso de integración y prueba de software

Grupo de Aseguramiento de Calidad (GAC)

Es un grupo independiente dentro de la organización que da soporte, realiza auditorías y asegura que los equipos cumplan con los estándares y procesos de calidad definidos.

Reglas

Tiene un conjunto de reglas innegociables para mantener su objetividad e independencia:

- **Independencia y Libertad:** No debe reportar al líder de proyecto, para mantener la independencia y la objetividad.
- **Jerarquía alta:** Debe tener un reporte independiente al de los proyectos, lo más cerca posible del nivel más alto de la organización. (No debe existir más de un nivel de gerencia entre la Dirección y el GAC.)
- **Interés Genuino:** Debe reportar a alguien (un sponsor) que esté realmente interesado en la calidad del software.

Modelos de Calidad

Son frameworks reconocidos internacionalmente que agrupan las mejores prácticas de la industria. Funcionan como una guía o estándar para que las organizaciones sepan que tienen que hacer para mejorar su forma de trabajo y evaluar si lo están haciendo bien.
(*dicen el "que" hay que cumplir, no el "cómo"*).
Evalúan la calidad del **proceso**, ya que la calidad del producto de software no se puede sistematizar con una plantilla universal.

CMMI (Capability Maturity Model Integration)

Modelo descriptivo (no prescriptivo) surgido para empresas de software, sirve como guía para mejorar la capacidad y madurez de los procesos organizacionales.

Se divide en tres dominios o constelaciones:

- **CMMI DEV:** Guia para procesos de desarrollo de software.
- **CMMI SVC:** Guia para entrega de servicios.
- **CMMI ACQ:** Guia para administrar y adquirir productos o servicios

Representación por Etapas

Mide la **madurez de la organización** en su conjunto. Provee una única clasificación que facilita las comparaciones entre empresas, dividiéndolas en inmaduras (*nivel 1*) y maduras (*nivel 2 al 5*):

- **Nivel 1 (Inicial):** Procesos impredecibles y el éxito depende del esfuerzo de los programadores ("Apagan incendios").

- **Nivel 2 (Administrado):** Existen controles básicos para la gestión de proyectos.

- **Nivel 3 (Definido):** Procesos estandarizados y documentados.

- **Nivel 4 (Cuantitativamente Administrado):** Se establecen mediciones cuantitativas para controlar el rendimiento.

- **Nivel 5 (Optimizado):** Foco exclusivo en la mejora continua y optimización.

Representación Continua

En lugar de medir a toda la empresa, mide la **capacidad de un área de proceso** en particular, en una escala de 0 a 5. Permite elegir áreas específicas a mejorar según las prioridades del negocio, sin tener que seguir una secuencia fija para toda la organización.

SPICE

Modelo de Mejora, son recomendaciones o estándares que se usan para encarar un proyecto interno cuyo resultado final será obtener un proceso mejorado para la organización.
Toman un proceso actual, que ya tiene una organización, y elaboran un proceso de transformación. El resultado será un proceso mejorado que se usará en los nuevos proyectos de desarrollo de la empresa.

ISO

- **ISO 9000 / 9001 (Para el Proceso):** Estándar de calidad teórica que fija pautas sobre cómo debe definirse el proceso de trabajo de la organización. Las organizaciones usan modelos de evaluación (auditorías de certificación) para medir el grado de adherencia de su proceso real a lo que dicta esta norma.

- **ISO 25010 (Para el Producto):** Es uno de los pocos modelos teóricos, apunta a evaluar las características de calidad del producto de software “en uso”.

Auditorías al Software

Son evaluaciones independientes y objetivas de los productos o procesos de software.Cómo son independientes, el auditor no debe depender jerarquía, funcional ni salarialmente del equipo evaluado.
El objetivo es asegurar el cumplimiento de estándares y procedimientos previamente establecidos.

Tipos de Auditorias

Auditoria de Proyecto

Su objetivo central es validar el cumplimiento del **proceso de desarrollo**, es decir, evalúa si el proyecto se ejecutó en base al proceso que el equipo se comprometio a usar.

- Se lleva a cabo de acuerdo a lo establecido en el **PACS** (Planeamiento de Aseguramiento de Calidad de Software), el cual debe indicar quien es la persona responsable de realizarla.

Auditorías de Configuración (Foco en el Producto)

Miran el producto (las baselines). Se dividen en dos, con un orden de ejecución estricto:

- **Auditoría de Configuración Física:** Hace **verificación.** Valida que el IC, tal como está construido, cumpla con la documentación técnica que lo describe. Si esta auditoría falla, la auditoría funcional no se realiza.

- **Auditoría de Configuración Funcional:** Hace **validación.** Valida que el producto cumpla con los requerimientos de la auditoría.

Auditorias Externas (GAC)

Evalúan al Grupo de SQA para validar si sus controles y procedimientos se ejecutan correctamente.

Roles en la auditoría

- **Gerente de SQA (El Grupo de Calidad):** Responsable de preparar el plan general de auditorías, calcula sus costos, asigna recursos necesarios y resuelve las no conformidades detectadas.

- **Auditor (EXTERNO):** Acuerda la fecha, comunica el alcance, recolecta y analiza la evidencia objetiva, ejecuta la auditoría, prepara el reporte y hace seguimiento de los planes de acción. (*Durante la ejecución debe escuchar sin interrumpir, manejar su lenguaje corporal y usar técnicas de cuestionario que empiezan con preguntas abiertas y cierran con preguntas cerradas*).

- **Auditado:** Participa proporcionando la evidencia solicitada, contesta el reporte, propone el plan de acción para arreglar las deficiencias y comunica cuando lo cumplió.

Proceso de Auditoría

1. **Preparación y planificación:** Se planifica conjuntamente entre el auditor y el auditado (no son “auditorías sorpresa”), estos acuerdan lugar, fechas, objetivos, participantes. Generalmente es convocada por el líder del proyecto.

2. **Ejecución:** Inicia con una **reunión de apertura** donde se explican la dinámica y los formalismos. Luego, el auditor realiza las entrevistas y recopila dos tipos de evidencias:
    - **Evidencia objetiva:** Documentación técnica, líneas base, registros y artefactos.
    - **Evidencia subjetiva:** Lo expresado verbalmente por los miembros del equipo sobre cómo ejecutan sus tareas.

    El rol del auditor: Debe escuchar sin interrumpir, observar y manejar el lenguaje corporal, tomar notas y utilizar una técnica de cuestionario que arranca con preguntas abiertas (qué, cómo, cuándo) y cierra con preguntas cerradas para clarificar.

    Reacciones comunes del auditado: Es normal que intente impresionar al auditor, se ponga ansioso, use el espacio para quejarse de la empresa, se enoje o dé información de más.

    Herramientas permitidas: *Checklist*, muestreo, revisión de registros y herramientas automatizadas.

- **Checklist (Listas de Control):** Es la herramienta por excelencia. Un cuestionario predefinido que asegura la cobertura total del control. Debe contener los datos del proyecto, el alcance y las preguntas clave (ej. "¿Existe un plan de proyecto actualizado?", "¿Están los requerimientos identificados unívocamente?").

3. **Análisis y reporte del resultado:** El auditor analiza las anotaciones y evidencias mediante una verificación cruzada contra el proceso definido. Se realiza una **reunión de cierre** para discutir los hallazgos y se entrega el reporte final.

   Se clasifican los resultados según:
   - **Buenas Prácticas:** Procedimientos o instrucciones que el equipo desarrolló mucho mejor de lo esperado.

   - **Desviaciones:** Incumplimientos del proceso que exige obligatoriamente la creación de un plan de acción por parte del auditado para ser corregidos.

   - **Observaciones:** Condiciones riesgosas que deberían mejorarse a futuro, pero que no son tan graves como para requerir un plan de acción formal.

4. **Seguimiento:** El auditor monitorea las desviaciones encontradas hasta considerar que han sido resueltas definitivamente por el equipo.
   El auditado diseña un plan de acción con fechas de compromiso para corregir las desviaciones. El auditor lo revisa, aprueba y verifica la efectividad de los cambios hasta dar el cierre formal a cada desviación

Calidad de Producto

El testing es un proceso destructivo cuyo objetivo NO es asegurar que el software funcione, sino encontrar defectos.
Por esto, un test solo es *exitoso* cuando se encuentra una falla.

Planificación y Proceso de Pruebas

El testing consume entre el 30% y el 50% del costo total del desarrollo. Como el testing exhaustivo (probar todos los caminos posibles) es inviable y muy caro, se debe planificar estratégicamente. El proceso tradicional de pruebas consta de:

1. **Planificación:** Crear y definir la estratégia del Plan de Prueba.

2. **Identificación y Especificación:** Definir detalladamente los Casos de Prueba (entradas y resultados esperados).

3. **Ejecución:** Correr las pruebas, ya sea de forma manual o automáticamente.

4. **Análisis de Fallas:** Evaluar si los resultados obtenidos coinciden con los esperados, en caso de no ser así, se corrige y repite el ciclo.

Niveles de Prueba

Son los “escalones” que se van subiendo a medida que el software crece, abarcando cada vez más componentes:

- **Pruebas Unitarias:** Prueban un componente individual y aislado. Las ejecuta el propio desarrollador, buscan encontrar *errores* (no defectos) y son las más fáciles de automatizar.

- **Pruebas de Integración:** Verifican el funcionamiento del conjunto de los componentes que ya pasaron las pruebas unitarias. Normalmente las ejecuta un *tester* y se hacen de manera incremental.

- **Pruebas de Sistema:** Evalúan el sistema en su totalidad. Buscan asegurar que todo funcione satisfactoriamente, incluyendo requerimientos funcionales y no funcionales. El ambiente debe ser casi idéntico al del producto.

- **Pruebas de Aceptación:** Ejecutadas por el cliente o el usuario final. Su meta principal no es encontrar defectos, sino establecer confianza en el sistema.

Tipos de Prueba

Clasifican el enfoque de lo que se está buscando validar en el sistema:

- **Smoke Test:** Es la primera corrida rápida de los test para verificar que las funcionalidades básicas “no se prendan fuego” antes de pasar a pruebas más exhaustivas.

- **Sanity Test:** Se hace después de una ronda exhaustiva o tras corregir bugs, para asegurar que los problemas críticos se arreglaron y que no se rompieron las funcionalidades claves.

- **Testing Funcional (Que hace el sistema?):** Evalúa las funciones específicas basándose en los requerimientos del cliente o procesos de negocio.

- **Testing No Funcional (Como lo hace?):** Evalúa atributos de calidad como rendimiento, Carga, Stress, Seguridad, Usabilidad y Portabilidad.

Técnicas y Herramientas (Estrategias de Prueba)

El objetivo de las estrategias es abarcar la mayor cantidad de software con el menor esfuerzo posible. Se dividen en dos enfoques complementarios:

Caja Blanca

El tester mira *adentro* del código y analiza la estructura interna del software para ver si algo dará un resultado inesperado.

El tester conoce la implementación técnica y revisa la estructura para verificar si algo dentro de esa definición arrojara un resultado no esperado.

Metodos

- **Cobertura de enunciados o caminos básicos:** Verifica que cada línea de código o ruta lógica se ejecute al menos una vez.
- **Cobertura de sentencias:** Asegura que todas las instrucciones individuales del código fuente sean evaluadas.
- **Cobertura de decisión:** Evalúa que las estructuras de control hayan tomado tanto el camino verdadero como el falso durante las pruebas.
- **Cobertura de condición:** Verifica de forma individual cada expresión booleana que compone una decisión compleja.
- **Cobertura múltiple:** Combina condiciones para probar todas las combinaciones lógicas posibles dentro del código.

Caja Negra

Se desconoce cómo está hecho el código internamente. Solo se evalúa si frente a determinadas entradas, el sistema arroja las salidas esperadas *(caso exitoso)*.
Se dividen en:

- **Basados en especificaciones:** Derivan directamente de los requerimientos.
    - *Partición de equivalencias:* Agrupa los datos de entrada en clases que deberán ser tratadas de la misma manera por el sistema, testeando solo un valor representativo de cada grupo para no hacer pruebas redundantes.
    - *Análisis de valores límites:* probar en las “esquinas” donde suelen agruparse los defectos (*más propensos*)
- **Basados en la experiencia:** Dependen del conocimiento y la intuición de un tester experimentado que sabe exactamente dónde ir para “romper” el sistema.
    - *Adivinanza de defectos:* El tester aplica su experiencia previa en sistemas similares para sabe dónde el programador pudo haberse equivocado.
    - *Testing Exploratorio:* Pruebas no estructuradas donde el tester interactúa libremente con la aplicacion para descubrir fallas ocultas que los casos de prueba formales no contemplaron.

TDD (Test Driven Development) | Herramienta Agil

Técnica de desarrollo donde las pruebas se escriben antes que el código de la funcionalidad.
Se basa en el ciclo de 3 pasos:

- **RED:** Escribir una prueba que inicialmente falla.
- **GREEN:** Escribir el código mínimo para que la prueba pase.
- **BLUE / REFACTOR:** Reestructurar el código para optimizarlo sin alterar su comportamiento.

Técnicas y Herramientas para la realización de revisiones técnicas del software.

Tecnica base (Peer Reviews)

Para el Aseguramiento de Calidad de Producto, la herramienta principal no es un software, sino un proceso metodológico

- **Evaluación entre Pares (Peer Reviews):** La técnica central consiste en que un colega de tu **mismo nivel técnico** revise tu trabajo. No debe ser un jefe ni un auditor externo.

- **Despersonalización del control:** La técnica exige separar el autor del producto; se revisa el código, el diseño o los requerimientos exclusivamente para buscar defectos de manera preventiva y mejorar la integridad interna, jamás para evaluar el desempeño de la persona.

Tecnicas de Ejecucion

A la hora de aplicar estas herramientas en una revisión de escritorio o entrevista de control, el evaluador debe dominar técnicas comunicacionales y conductuales:

- **Tecnica de Cuestionario de Embudo:** Se debe arrancar con preguntas de final abierto para que el evaluado se explaye, hacer preguntas cortas en el medio, y terminar con preguntas de final cerrado para clarificar conceptos.

- **Manejo Conductual:** El revisor / auditor debe escuchar sin interrumpir, controlar su propio lenguaje corporal y saber las reacciones del evaluado, quien a menudo puede estar a la defensiva, tenso o intentando dar respuestas para “impresionar” en lugar de mostrar la realidad del proyecto

Testing en Ágile

Se apoya en el “Manifiesto de Testing Ágil”.
Su pilar central es que **no se hace testing al final, sino de forma continua mientras se construye el software.**
La calidad se asume como una ***responsabilidad compartida por todo el equipo***, eliminando la idea de que recae únicamente en un rol exclusivo de tester.

Manifiesto de Testing Ágil

Prioriza ciertas prácticas sobre otras, buscando cambiar la mentalidad del tester:

- Testear de forma continua por sobre testear al final del proyecto.
- Prevenir errores por sobre encontrar defectos (el enfoque preventivo evita el retrabajo).
- Testear el entendimiento por sobre verificar la funcionalidad (darle lógica a lo que se prueba, no solo tildar casillas).
- Construir el mejor sistema por sobre intentar romper el sistema.

- La calidad es responsabilidad de todo el equipo por sobre la responsabilidad exclusiva del tester (en ágil, no hay un rol de tester fijo, cualquiera puede y debe probar).

Principios

1. El testing se mueve hacia adelante en el proyecto (no queda para el final, se hace constantemente en cada iteración).
2. El testing no es una fase, se hace mientras se desarrolla el código.
3. Todos hacen testing (no hay un rol exclusivo).
4. Reducir la latencia del feedback.
5. Las pruebas representan expectativas.
6. Mantener el código limpio y corregir los defectos rápido (prevenir en vez de buscar acumular bugs).
7. Reducir la sobrecarga de documentación en las pruebas.
8. Las pruebas son parte del criterio de “Done”.
9. Pasar de probar al final a un desarrollo “conducido por pruebas”.

Practicas

- **TDD (Test Driven Development):** Escribir las pruebas unitarias antes de implementar el código funcional. Sigue el ciclo:
    - ***RED:*** Escribir una prueba que inicialmente falla.
    - ***GREEN:*** Escribir el código mínimo para que la prueba pase.
    - ***BLUE / REFACTOR:*** Reestructurar el código para optimizarlo sin alterar su comportamiento.
- **ATDD (Acceptance TDD):** Desarrollo conducido por pruebas de aceptación definidas conjuntamente entre el equipo y el negocio.
- **Automatización de Pruebas:** Automatizar todo lo que sea repetitivo o mecánico (pruebas unitarias, de integración y regresión a nivel de sistema) para liberar tiempo del equipo y acelerar la entrega.
- **Testing Exploratorio:** Pruebas manuales basadas en la experiencia, intuición y creatividad del tester, no pueden automatizarse.
- **Control de Versión Conjunto:** Los casos de prueba (Unit Test y System Tests) se gestionan y guardan en el mismo repositorio junto con el código fuente.

Cuadrantes del Testing Ágil

Clasifican los tipos de pruebas según su orientación (Tecnologia vs Negocio) y su objetivo (soporte al equipo vs Critica al producto):



- **Cuadrante 1 (Tecnológico / Soporte al equipo):** Son automatizadas y las ejecuta el equipo de desarrollo.

- **Cuadrante 2 (Negocio / Soporte al equipo):** Pueden ser manuales o automatizadas.

- **Cuadrante 3 (Negocio / Critica del producto):** Son de ejecución manual.

- **Cuadrante 4 (Tecnológico / Crítica del producto):** Se realizan con herramientas automatizadas específicas.